

课程报告四：基于神经网络的《九章算术》语言模型与文本嵌入

方俊睿

学号：3230104069

仓库地址：<https://github.com/Tomabsolute/Neul.git>

1 实验背景介绍

本实验面向《九章算术》文本训练小型神经网络语言模型，目标不是构建通用大语言模型，而是在课程规模内观察模型是否能够从古代数学文本中学习到稳定的题目、答案和数学词汇模式。附件文本包含题目与「答曰」后的解答。程序预处理时将原文中的异体写法统一为「答曰」，再抽取题面-答案对用于生成模型训练与评估。

实验包含两部分：第一部分为序列生成模型，采用字符级 LSTM 语言模型，在给定题面和「答曰：」提示后生成答案文本；第二部分为文本嵌入模型，采用 Skip-gram Negative Sampling 训练轻量 word2vec 式嵌入，通过近邻词观察模型是否捕捉到《九章算术》中的数学概念关系。

2 实验任务与目标

本实验的具体目标如下：

- 从原始《九章算经》文本中自动识别题目与答案；
- 训练一个字符级 LSTM 序列生成模型，生成「答曰」后的答案；
- 训练一个文本嵌入模型，展示数学词汇或字符的相似关系；
- 记录训练损失、困惑度、答案关键词命中率与生成样例；
- 提供可在服务器上直接运行的脚本与报告。

3 数据集与预处理

3.1 文本读取

原始文件为 work4/九章算经.txt。该文件编码方式为 GB 2312，因此读取函数会依次尝试 utf-8、gb2312、gb18030 与 gbk。文本读取后进行以下标准化：

- 将 Windows 换行统一为 \n；
- 将原文中的异体写法统一为「答曰」；
- 合并多余空格和连续空行；
- 训练时按字符切分，去掉空白符。

3.2 题答对抽取

《九章算术》题目通常以「〔一〕」「〔二〕」等编号开头，答案由「答曰：」引出。脚本使用正则表达式切分编号块，并在每个块中查找「答曰：」。若找到答案，则构造

输入提示 = 题面 + “答曰：”， 目标输出 = 答案文本。

抽取结果按随机种子划分为训练集和验证集，其中验证集比例为 15%。这种划分能够检验模型对未见题目的续写能力。

4 序列生成模型

4.1 模型结构

序列生成部分采用字符级 LSTM。选择字符级建模的原因是《九章算术》包含大量古汉语数字、单位和术语，直接按字建模可以避免分词工具对繁体古文不稳定的问题。模型结构如下：

表 1: 字符级 LSTM 语言模型结构

模块	默认配置
输入	字符 ID 序列
Embedding	128 维
LSTM	2 层, 隐藏维度 256
Dropout	0.2
Linear	隐藏状态映射到词表大小

训练时将每一条题答对作为一个样本, 输入为「题面 + 答曰: + 答案」。为了让模型更关注答案生成, 损失只在答案部分计算, 题面部分只作为条件上下文。损失函数为交叉熵:

$$\mathcal{L} = - \sum_t \log p(x_{t+1} | x_{\leq t}).$$

推理时输入完整题面和「答曰:」, 模型逐字符采样生成答案, 遇到结束标记后停止。

4.2 训练配置

表 2: LSTM 默认训练配置

项目	配置
训练文本	抽取出的题面-答案对
序列长度	256
Batch size	64
Epoch	150
优化器	AdamW
学习率	2×10^{-3}
梯度裁剪	1.0
设备	CUDA / MPS / CPU 自动选择

4.3 评价方式

语言模型的数值指标包括验证集交叉熵和困惑度。为了更直接观察模型是否学到知识, 脚本还在验证题目上生成答案, 并计算答案关键词命中率。关键词主要来自数字和单位, 例如「一、二、三、十、百、分、步、畝、頃、尺、升、斗」等。若生成文本覆盖至少一半答案关键词, 则记为一次命中。

5 文本嵌入模型

5.1 模型结构

文本嵌入部分实现 Skip-gram Negative Sampling。给定中心字符 w_t , 模型预测窗口内上下文字符 w_c , 并用负采样近似 softmax。损失为:

$$\mathcal{L} = -\log \sigma(v_{w_t}^\top u_{w_c}) - \sum_{i=1}^K \log \sigma(-v_{w_t}^\top u_{n_i}).$$

其中 K 为负样本数, 默认取 8。训练完成后使用输入嵌入向量, 并通过余弦相似度检索近邻。

5.2 训练配置

表 3: Skip-gram 默认训练配置

项目	配置
Token 粒度	字符
Embedding 维度	128
窗口大小	4
负样本数	8
Batch size	2048
Epoch	25, 可在服务器上调大
优化器	AdamW

6 代码与运行方式

实验代码位于 `work4/`:

- `train_lm.py`: 训练 LSTM 语言模型;
- `generate_answer.py`: 加载模型并生成答案;
- `train_word2vec.py`: 训练 Skip-gram 文本嵌入;
- `run_all_experiments.sh`: 一键运行两个实验;
- `src/`: 数据处理、模型和画图工具。

一键运行命令为:

```
bash work4/run_all_experiments.sh
```

服务器上可以使用:

```
LM_EPOCHS=150 EMB_EPOCHS=40 BATCH_SIZE=64 \
  bash work4/run_all_experiments.sh
```

7 实验结果展示

服务器实验使用 CUDA 训练, LSTM 训练 150 轮, Skip-gram 训练 40 轮。训练完成后, LSTM 语言模型输出 `metrics.json`、`history.csv`、`generation_samples.md` 和训练曲线; 嵌入模型输出 `neighbors.md`、`neighbors.json` 与近邻可视化图。

表 4: LSTM 语言模型服务器训练结果

项目	数值
题答对数量	452
训练 / 验证题答对	385 / 67
词表大小	451
最优验证 epoch	147
最优验证 loss	1.1857
最优验证 PPL	3.2730
答案关键词命中率	0.6333
训练 epoch	150
LSTM 参数规模	约 1-2M, 随词表略变

表 5: Skip-gram 嵌入模型服务器训练结果

项目	数值
Token 数量	59239
训练样本对	473892
词表大小	1046
Embedding 维度	128
窗口大小	4
负样本数	8
训练 epoch	40
最终 loss	2.3327

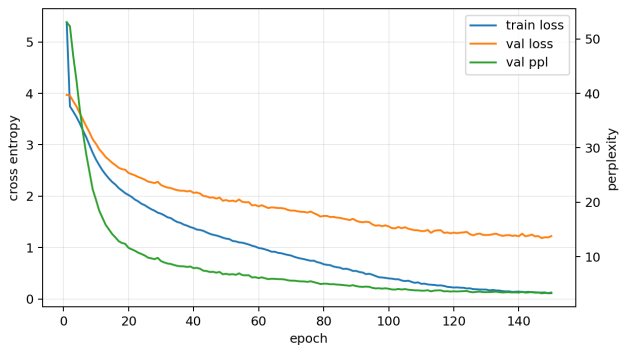


图 1: 字符级 LSTM 训练曲线

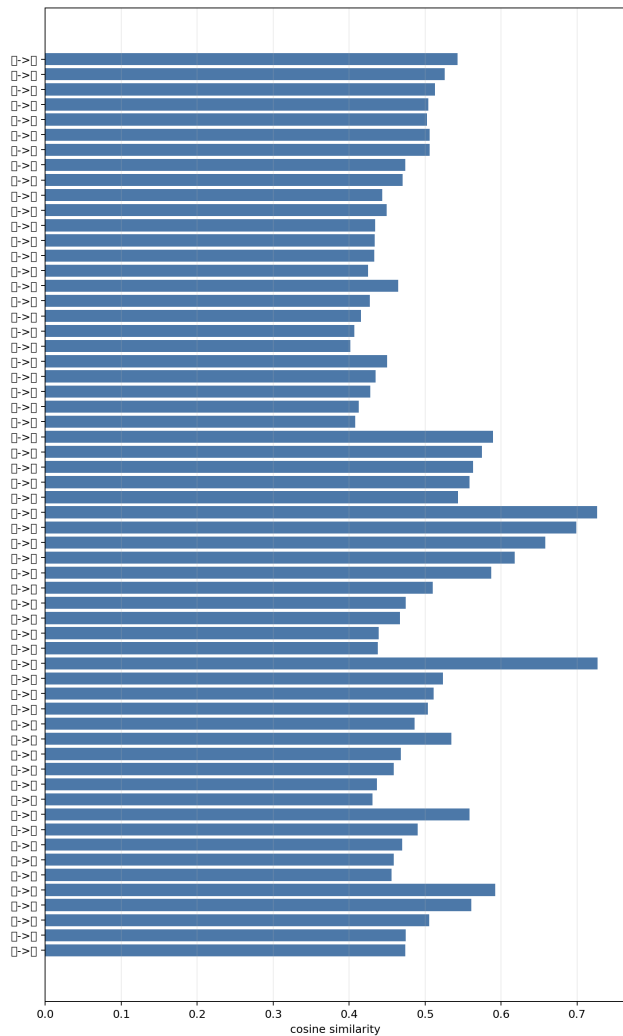


图 2: Skip-gram 嵌入近邻相似度

7.1 生成结果分析

训练曲线显示, 监督式题答训练后验证集损失在第 147 轮达到最低, 最终保存验证损失最低的 `best_lstm_lm.pt`。在手动测试题「今有田廣十五步, 從十六步」上, 模型生成「一畝」, 与原书答案一致, 说明模型能复现部分典型题型。

表 6: LSTM 生成样例

题型	标准答案	生成答案
田廣十五步, 從十六步	一畝	一畝
圓形土堡, 周四丈八尺, 高一丈一尺	二千一百一十二尺	二千一百一十二尺
约分: 九十一分之四十九	十三分之七	十三分之七
股四尺, 弦五尺, 求句	三尺	三尺
井徑五尺, 不知其深	五丈七尺五寸	五丈二尺

从样例看，模型已经学习到《九章算术》的答案格式、数字表达和单位体系，例如「畝」「尺」「寸」「分之」等模式会稳定出现；部分短题和高频题型能够生成完全正确的答案，例如方田、约分和勾股样例。但模型仍不能可靠掌握精确算术推理，复杂均输、粮食换算和多变量问题中经常出现单位相关但数值错误的答案。关键词命中率为 0.6333，只能说明模型在部分验证样例中生成了相关数字或单位，不能等同于算术正确率。

7.2 嵌入结果分析

文本嵌入的近邻可以从另一个角度验证学习效果。服务器结果中，「粟」的近邻包含粗米、饭、稻、米等粮食换算相关字，「米」的近邻也集中在饭食和米制品语境中，说明模型较好捕捉到粟米章中的共现关系；「尺」的近邻包含「寸、丈、深、竿、绳」一类长度测量相关字，也符合勾股、井深类题目的语境。「田」的近邻中出现「頃」，「步」的近邻中出现「田」，说明面积单位关系也有一定体现。部分近邻如「惡、箕、圭」等不够理想，主要原因是字符级模型容易受到古文上下文和低频字共现干扰。

8 不足与改进方向

- 本实验使用字符级模型，能处理古文编码和繁体字，但长距离推理能力有限；
- LSTM 主要学习文本模式，不能保证精确执行算术运算；
- 关键词命中率是弱指标，后续可人工构造更多保留题型但改变数字的测试题；
- 若服务器资源充足，可替换为 Transformer Decoder 或 Mamba，并增加上下文长度；
- 文本嵌入当前为字符级，可进一步加入基于规则的古汉语词表，训练词级 fastText。

9 结论

本实验完成了一个可复现的《九章算术》神经语言模型实验。字符级 LSTM 用于从题面续写「答曰」后的答案，Skip-gram 负采样模型用于学习数学文本嵌入。训练脚本会自动处理原始附件编码、抽取题答对、保存模型和结果图表。最终通过困惑度、答案关键词命中率、生成样例和嵌入近邻共同展示模型是否学习到《九章算术》中的知识。